Étude et analyse de modèles simulant une trajectoire circulaire

Manuel Rocca, Damian Kazberuk, Matteo Morbée, Romain Liefferinckx, Martin Gouverneur

April 25, 2026

Abstract

Réalisé à l'attention des professeurs et assistants de la faculté des sciences de l'ULB.

Introduction

La modélisation et la simulation de phénomènes physiques constituent des outils fondamentaux en sciences, permettant de décrire, analyser et prédire le comportement de systèmes dynamiques. Dans de nombreux contextes, plusieurs approches de modélisation peuvent être envisagées, reposant sur des cadres théoriques distincts et impliquant des compromis entre précision, complexité et coût de calcul. Le présent travail s'inscrit dans ce cadre et porte sur l'étude de différentes méthodes de simulation appliquées à un système physique simple: la trajectoire d'une bille en mouvement circulaire sur un dispositif expérimental. Bien que ce système soit relativement élémentaire, il présente une dynamique suffisamment riche pour mettre en évidence les avantages et les limites de différentes approches de modélisation. Deux grandes familles de méthodes sont considérées.

La première regroupe les modèles fondés sur des équations physiques explicites, dans lesquels la dynamique du système est décrite à l'aide de lois fondamentales. Ces approches offrent une interprétation directe des phénomènes observés, mais nécessitent l'introduction d'hypothèses simplificatrices et peuvent devenir difficiles à exploiter analytiquement ou numériquement lorsque la complexité du système augmente.

La seconde famille repose sur des techniques d'apprentissage automatique, visant à approximer la dynamique du système à partir de données. Dans ce cadre, le problème est reformulé comme l'apprentissage d'une application reliant les conditions initiales du système à son évolution temporelle.

Ces méthodes permettent de capturer des comportements non linéaires complexes sans expliciter les lois physiques sous-jacentes, au prix d'une dépendance accrue à la qualité des données et d'une interprétabilité plus limitée. Au-delà de la construction des modèles, une attention particulière est portée à l'analyse des erreurs affectant les simulations. Celles-ci incluent à la fois les erreurs de mesure, les erreurs de modélisation résultant des hypothèses adoptées, ainsi que les erreurs numériques introduites par les méthodes de calcul. L'étude de propriétés telles que la consistance, la stabilité et le conditionnement permet ainsi d'évaluer la robustesse et la fiabilité des approches considérées. L'objectif de ce rapport est d'analyser et de comparer ces différentes méthodes de simulation, en mettant en évidence leurs performances, leurs limites et leur domaine de validité, tant du point de vue physique que numérique.

Modèles

Modèle conique

Le modèle conique constitue une première approximation de la géométrie de la membrane, reposant sur l'hypothèse d'une pente radiale constante. Cette approche revient à assimiler la surface à un cône de révolution, ce qui permet de simplifier considérablement l'expression des forces en jeu. Bien que rudimentaire, ce modèle fournit un cadre analytique simple permettant d'interpréter qualitativement la dynamique du système.

Équations

Dans ce modèle, la pente radiale de la surface est supposée constante :

$$\frac{dz}{dr} = \tan(\alpha)$$

où α désigne l'angle d'inclinaison de la surface.

L'intégration de cette relation conduit à l'expression de la surface :

$$z(r) = r\tan(\alpha) + C$$

où C est une constante fixant le niveau de référence.

La composante radiale de l'accélération, induite par la pente de la surface, s'écrit :

$$a_r = g\left|\frac{dz}{dr}\right| = g\tan(\alpha)$$

Il en résulte une accélération constante, indépendante de la position radiale.

Limitations et hypothèses

Ce modèle repose sur des hypothèses fortement simplificatrices :

- La pente de la surface est uniforme, ce qui exclut toute variation de courbure et ne permet pas de rendre compte de la géométrie réelle de la membrane.
- Les effets liés à une éventuelle concentration de masse ou de contrainte au voisinage du centre sont négligés.
- La dynamique résultante est linéarisée, l'accélération étant indépendante de la position.
- Ce modèle ne permet pas de décrire des phénomènes non linéaires ou des régimes dynamiques complexes.

En conséquence, le modèle conique doit être considéré comme une approximation de premier ordre, pertinente pour des analyses qualitatives ou comme cas de référence.

Modèle de Laplace

Le modèle de Laplace vise à décrire de manière plus réaliste la déformation de la membrane en introduisant une dépendance radiale de la pente. Il s'inspire de systèmes physiques gouvernés par des interactions centrales, dans lesquels l'intensité du champ varie avec la distance à une source localisée. Cette approche permet de capturer des effets non linéaires absents du modèle conique.

Équations

Dans ce cadre, la pente de la surface est donnée par :

$$\frac{dz}{dr} = \frac{K}{mgr^2}$$

où K est une constante caractéristique du système, m la masse de la bille, g l'accélération de la pesanteur et r la distance au centre.

L'intégration de cette expression conduit à :

$$z(r) = -\frac{K}{mgr} + C$$

La composante radiale de l'accélération s'écrit alors :

$$a_r = g\left|\frac{dz}{dr}\right| = \frac{K}{mr^2}$$

Cette dépendance en $1/r^2$ est caractéristique des champs centraux et induit une dynamique fortement non linéaire.

Afin de pallier la divergence de la pente au voisinage du centre, une version régularisée du modèle est introduite :

$$\frac{dz}{dr} = \begin{cases} \frac{K}{mgr^2} & \text{si } r \geq r_c \\ \frac{K}{mgr_c^2} & \text{si } r < r_c \end{cases}$$

où r_c désigne un rayon de coupure. Cette construction assure la continuité de la pente au point de raccordement.

Limitations et hypothèses

Le modèle de Laplace repose sur plusieurs hypothèses structurantes :

- La source centrale est assimilée à une masse ponctuelle, ce qui conduit à une singularité en $r = 0$ dans la formulation non régularisée.
- La régularisation introduite via le paramètre r_c constitue une correction ad hoc, motivée par des considérations physiques et numériques.
- Bien que la pente soit continue, la dérivée seconde de la surface présente une discontinuité au point $r = r_c$, traduisant une rupture de courbure.
- La forte variation de la pente à proximité du centre peut engendrer des difficultés numériques, notamment en termes de stabilité et de précision des schémas d'intégration.
- Le modèle ne prend pas en compte d'éventuels effets dissipatifs ou des déformations complexes de la membrane.

Malgré ces limitations, ce modèle permet de rendre compte de comportements dynamiques plus riches que le modèle conique, en particulier ceux associés à des interactions de type champ central. Il constitue ainsi une approximation plus fidèle du système étudié, au prix d'une complexité accrue.

Modèle basé sur du machine learning (ML)

Une seconde classe de méthodes de simulation repose sur l'utilisation de techniques d'apprentissage automatique, visant à approximer directement la dynamique du système à partir de données. Contrairement aux approches déterministes fondées sur la modélisation explicite des lois physiques, ces méthodes s'inscrivent dans un cadre empirique, dans lequel la relation entrée-sortie est apprise à partir d'un ensemble fini d'observations. Dans cette étude, les données générées par les simulations physiques sont utilisées comme ensemble

d'entraînement. L'objectif du ML est d'apprendre une fonction

$$f : \mathbb{R}^4 \to \mathbb{R}^{2N}$$

telle que:

$$\mathbf{y} = f(\mathbf{x}),$$

où $\mathbf{x}$ représente les conditions initiales du système et $\mathbf{y}$ la trajectoire correspondante.

Plus précisément, le vecteur d'entrée est défini par:

$$\mathbf{x} = (x_0, y_0, v_{x0}, v_{y0}) \in \mathbb{R}^4,$$

où x_0 et y_0 désignent les coordonnées initiales de la bille, et v_{x0}, v_{y0} les composantes initiales de sa vitesse.

La sortie correspond à une discrétisation temporelle de la trajectoire:

$$\mathbf{y} = (x_1, y_1, x_2, y_2, \ldots, x_N, y_N) \in \mathbb{R}^{2N},$$

où N désigne le nombre de pas de temps considérés. Cette formulation permet d'exprimer la dynamique du système sous la forme d'une application entre vecteurs d'entrée et de sortie.

Deux modèles ont été considérés pour approximer la fonction f : la régression linéaire (LR) et le perceptron multicouche (MLP).

Regression linéaire La régression linéaire constitue un modèle paramétrique dans lequel la sortie est exprimée comme une combinaison affine des variables d'entrée. Dans notre cas, elle est utilisée pour approximer la fonction $f : \mathbb{R}^4 \to \mathbb{R}^{2N}$ reliant les conditions initiales à la trajectoire de la bille.

Plus précisément, le modèle s'écrit :

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

où $\mathbf{W} \in \mathbb{R}^{2N \times 4}$ est une matrice de paramètres et $\mathbf{b} \in \mathbb{R}^{2N}$ un vecteur de biais.

Chaque composante de la trajectoire prédite est ainsi modélisée comme une combinaison linéaire des conditions initiales. Cette approche revient à supposer que la dynamique du système peut être approximée globalement par une transformation affine.

Toutefois, un tel modèle ne permet pas de capturer des dépendances temporelles explicites entre les différents instants de la trajectoire, ni des comportements non linéaires. Par conséquent, sa capacité de modélisation reste limitée pour des dynamiques complexes.

Multicouche perceptron Afin de modéliser des comportements plus complexes, un perceptron multicouche a également été utilisé. Dans le cas d'une architecture à une couche cachée, la fonction f s'écrit :

$$f(\mathbf{x}) = \mathbf{W}^{(2)}\, \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)},$$

où σ est une fonction d'activation non linéaire. σ est représentée par la fonction ReLU (Rectified Linear Unit), définie par $\sigma(z) = \max(0, z)$.

Les paramètres du modèle, regroupés dans θ, sont estimés en minimisant une fonction de coût, généralement choisie comme l'erreur quadratique moyenne (MSE) :

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{i=1}^{M} \left\| \mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)} \right\|_2^2,$$

où M désigne le nombre d'échantillons d'entraînement. Pour chaque exemple i, $\mathbf{y}^{(i)}$ correspond à la trajectoire réelle associée aux conditions initiales $\mathbf{x}^{(i)}$, et $\hat{\mathbf{y}}^{(i)} = f_\theta(\mathbf{x}^{(i)})$ la prédiction du modèle.

Valabilité des modèles de ML

Erreurs de modélisation

La catégorie des erreurs de modélisation se décline en deux types d'erreurs.

Le premier type, les erreurs de mesure qui se produisent dans le cadre physique de notre expérience. En effet, nous ne disposons pas d'outils de mesure parfaits, ce qui induit donc des imprécisions numériques de l'ordre du millimètre lorsque nous mesurons des éléments de notre expérience comme par exemple la largeur du dispositif.

Le second regroupe les erreurs de modélisation du phénomène; des erreurs qui apparaissent lorsque nous faisons des simplifications, des hypothèses dans notre modèle pour le rendre plus facile à comprendre ou à résoudre. Un exemple d'hypothèse émise dans le cadre de notre expérience concerne la pression de l'air, supposée sans impact sur la trajectoire de la balle.

Calcul numérique

Erreurs numériques

Dans le cadre d'une approche scientifique, il est important de prendre en compte tous les paramètres pouvant entrer en jeu et potentiellement perturber et modifier l'expérience. Nous avons déjà abordé les erreurs de modélisation mais ce ne sont pas les seules entrant en jeu. En particulier, nous considérons les erreurs numériques qui apparaissent lorsque l'ordinateur effectue des calculs [1, p. 16]. Elles se déclinent sous-différentes, chacune explicitée dans les sous-sections suivantes.

Erreurs de troncature Nos modèles itératifs à pas discret sont infinis. Il est donc impossible d'obtenir une solution exacte en un temps fini même en faisant abstraction de tout autre type d'erreur pouvant entrer

en jeu. Afin d'obtenir un résultat, nous établissons une variable *n_steps* qui limite le nombre d'itération de manière à obtenir un bon équilibre entre temps d'exécution et précision du résultat.

Erreurs d'arrondi Les erreurs d'arrondi sont des erreurs causées par la représentation finie des nombres en machine dans les registres du processeur (généralement 32 ou 64 bits). L'ordinateur ne peut pas représenter tous les nombres réels avec une précision infinie, ce qui crée alors une approximation.

Pour définir formellement ce type d'erreur, notons par $\mathbb{F}(b, t, L, U)$ l'ensemble des nombres réels qui sont représentés par une notation à virgule flottante en base b, comportant t chiffres significatifs et dont l'exposant varie dans l'intervalle $[L, U]$. Considérons un ordinateur utilisant cette notation $\mathbb{F}$. Soit $x \in \mathbb{R}$ un nombre réel quelconque et $fl(x) \in \mathbb{F}$ la représentation machine à virgule flottante de x. Désignons par ε_x son erreur relative

$$\varepsilon_x = \frac{|fl(x) - x|}{|x|} \tag{1}$$

Nous allons évaluer la borne supérieure de l'erreur relative d'arrondi. Il est évident que cette quantité ne peut dépasser la moitié de la distance relative entre les deux nombres consécutifs à virgule flottante $x_i \in \mathbb{F}(b, t, L, U)$ et $x_{i+1} \in \mathbb{F}(b, t, L, U)$ tels que

$$x_i \leq x \leq x_{i+1} \tag{2}$$

Il vient alors que

$$\varepsilon_x \equiv |\rho_x| \leq \frac{1}{2}\left|\frac{x_{i+1} - x_i}{x}\right| \leq \frac{1}{2}b^{1-t} = \frac{1}{2}\epsilon = u, \tag{3}$$

où u est dite la précision machine.

Avec la notion d'écart relatif définie telle que

$$\rho_x = \frac{\hat{x} - x}{x} \tag{4}$$

nous obtenons

$$\hat{x} = fl(x) = x(1 + \rho_x), \tag{5}$$

où $\varepsilon_x = |\rho_x| \leq u$ est dite l'erreur d'arrondi [1].

Il est clair que nous retrouvons ce type d'erreur dans nos simulation.

Erreurs de propagation et génération Les erreurs de propagation et de génération sont des erreurs qui apparaissent lorsque les erreurs numériques se propagent à travers les calculs. Les erreurs de génération se produisent à chaque itération, étape des calculs et les erreur de propagation regroupent toutes les erreurs de génération jusqu'à l'étape courante. Par exemple, si nous simplifions π à 3.14, nous introduisons une erreur d'arrondi à une certaine génération qui va se propager à travers les calculs ultérieurs, affectant ainsi la précision globale des résultats.

Analyse numérique du problème

Nous étudions, dans les sous-sections suivantes, les notions de consistance, stabilité et de conditionnement afin d'analyser les propriétés de nos modèles. Par ces analyses, nous pouvons prédire certains comportements de nos modèles lors des simulations.

Définissons d'abord la notion d'algorithme numérique. Étant donné un problème bien posé $F(x, d) = 0$, nous définissons l'algorithme numérique pour la résolution du problème F par la suite de problèmes approchés

$$F_1\big(x^{(1)}, d^{(1)}\big),\ F_2\big(x^{(2)}, d^{(2)}\big),\ \ldots,\ F_n\big(x^{(n)}, d^{(n)}\big) \tag{6}$$

dépendant d'un paramètre n, où $x^{(n)}$ est la solution du sous-problème F_n [1].

Consistance

La consistance traduit la capacité d'un modèle numérique à tendre vers les lois pures de la physique lorsque le pas de temps s'affine. Formellement [1], un schéma $F_n(x^{(n)}, d^{(n)})$ est consistant si la solution exacte x vérifie le modèle limite :

$$\lim_{n\to\infty} F_n(x, d^{(n)}) = F(x, d) = 0 \tag{7}$$

En pratique, pour une trajectoire $y' = f(t, y)$, la méthode d'Euler (explicite ou semi-implicite) admet le développement de Taylor suivant :

$$y(t_{n+1}) = y(t_n) + \Delta t f\big(t_n, y(t_n)\big) + \mathcal{O}(\Delta t^2) \tag{8}$$

L'erreur locale de troncature par pas de temps est $\tau_{n+1} = \mathcal{O}(\Delta t)$. Ces méthodes sont donc consistantes à l'ordre 1. À l'inverse, l'approche de Runge-Kutta (RK4) annule analytiquement les termes d'erreur d'ordre inférieur, offrant $\tau_{n+1} = \mathcal{O}(\Delta t^4)$, soit une très haute consistance polynomiale.

Stabilité

La stabilité garantit que les perturbations inévitables (comme les erreurs d'arrondi) ne s'amplifient pas exponentiellement. Un algorithme est numérique stable si, pour tout $\epsilon > 0$, il existe $\eta > 0$ tel que toute perturbation initiale d'amplitude η affecte la solution finale de moins de ϵ.

Pour l'analyser, on utilise couramment l'équation de test linéaire $y' = \lambda y$ (avec $\lambda \in \mathbb{C}$). Pour l'Euler explicite, sa récurrence dicte $y_{n+1} = G \cdot y_n$, avec $G = (1 + \lambda\Delta t)$. La stabilité exige impérativement $|G| \leq 1$. Cependant, pour notre trajectoire purement inertielle (système conservatif), λ est imaginaire pur ($\lambda = i\omega$). Le facteur d'amplification devient :

$$|G| = |1 + i\omega\Delta t| = \sqrt{1 + \omega^2\Delta t^2} > 1 \tag{9}$$

L'Euler explicite amplifie artificiellement l'énergie du système oscillatoire et diverge de son orbite. En structurant l'intégration différemment, l'Euler semi-implicite met à jour la position depuis la vitesse future. Cette conception *symplectique* préserve le volume de l'espace des phases et assure une grande stabilité à long terme pour les orbites.

Convergence et Machine Learning

Le **théorème d'équivalence de Lax** stipule logiquement que pour un problème bien posé, la consistance couplée à la stabilité assure la **convergence** globale du modèle discret vers la vérité continue.

Toutefois, ce théorème ne s'extrapole pas aux réseaux de neurones (MLP, LR). Entraînés de manière isolée pour lisser l'étape $n \to n+1$, la régression est aveugle aux variations sur temps long. Intégrée en boucle auto-régressive (*step-by-step*), ses légères imprécisions déplacent l'état évalué hors du domaine vu à l'apprentissage (un *distribution shift*). Ces erreurs alimentent exponentiellement les futures imprécisions, déclenchant sur la durée une déviation divergente similaire à de l'instabilité numérique classique.

Conditionnement

Contrairement à la stabilité qui évalue l'algorithme choisi, le conditionnement diagnostique la susceptibilité intrinsèque du *problème analytique* en lui-même. Pour une évolution de type $y' = f(t, y)$, la variation $\delta y(t)$ issue d'une erreur d'entrée se dilate proportionnellement à la matrice Jacobienne $J = \frac{\partial f}{\partial y}$:

$$\delta y' = J\delta y \implies \|\delta y(t)\| \propto \exp\left(\int \|J\| dt\right) \tag{10}$$

Dans le système circulaire abordé ici, les valeurs propres de la Jacobienne ont une composante réelle négligeable : l'erreur n'y grimpe pas de façon irrémédiable, et on caractérise ce problème comme globalement **bien conditionné**. Toutefois, la manipulation informatique y imbrique de permanentes erreurs d'arrondi (dues à la précision machine limitée $u = \frac{1}{2}b^{1-t}$). Ce contexte bien conditionné n'évitera ainsi pas la perte d'exactitude si un schéma divergent (comme l'Euler explicite) est forcé sur la simulation.

Dans le registre Machine Learning, le conditionnement trouve son équivalent dans la constante de Lipschitz de la fonction de mappage f_θ. Sans régularisation plafonnant la norme de pointe spectrale (les poids $\|\mathbf{W}\|$), l'équation d'état affine $\hat{\mathbf{y}}_{n+1} = \mathbf{W}\hat{\mathbf{y}}_n + \mathbf{b}$ se révèle particulièrement volatile et mal conditionnée au moindre bruit incident.

Comparaison des méthodes d'approximation numérique

Plusieurs méthodes d'approximation numérique peuvent être utilisées pour résoudre les équations différentielles ordinaires associées aux différents modèles étudiés. Le tableau suivant présente, pour chaque méthode et chaque modèle, l'erreur géométrique normalisée mesurant l'écart entre la trajectoire simulée et la trajectoire réelle.

Table 1: *Erreur géométrique normalisée entre la trajectoire simulée et la trajectoire réelle pour chaque combinaison de modèle et de méthode numérique.*

Méthode	Conique	Laplace	Laplace R_C
Euler exp.	0.7681	0.7682	0.7680
Euler semi-imp.	0.7681	0.7682	0.7680
RK4	0.7682	0.7682	0.7680

Discussion et conclusion

Conclusion et discussion

Dans le cadre de ce travail, la comparaison des différents modèles de simulation ne peut se limiter à un unique critère de performance. En effet, les approches considérées présentent des caractéristiques hétérogènes, tant du point de vue de la fidélité physique que des propriétés numériques ou de leur mise en œuvre. Il apparaît ainsi nécessaire d'introduire un cadre d'analyse permettant de confronter ces modèles selon plusieurs dimensions pertinentes.

À cette fin, une démarche d'évaluation multicritère est adoptée. Celle-ci repose sur l'identification de quatre axes principaux :

- la précision des résultats, évaluée à travers les erreurs numériques (35%)
- les hypothèses simplifiantes considérées et leur quantité (35%)
- le temps requis pour effectuer les simulations (10%)
- l'accessibilité du modèle, incluant à la fois sa simplicité de mise en œuvre au niveau technique et sa capacité à être interprété ou vulgarisé (20%)

En attribuant les notes correspondantes à chaque modèle selon ces critères, nous obtenons:

Une telle approche permet de construire un indicateur synthétique facilitant la comparaison globale des modèles, tout en conservant une lecture structurée des compromis qu'ils impliquent. Elle s'inscrit dans une logique d'aide à la décision plutôt que dans une volonté d'établir un classement absolu.

Nous tenons toutefois à souligner que cette pondération repose en partie sur des choix méthodologiques propres à ce travail subjectifs. Elle a pour but de regrouper tous les aspects rencontrés au cours de notre travail, notamment l'aspect de vulgarisation qui représentait un aspect majeur lors de notre présentation au Printemps des Sciences. Dans ce sens, l'évaluation proposée doit être comprise comme qualitative et indicative: elle vise à structurer l'analyse et à expliciter les compromis entre modèles, sans prétendre à une objectivité stricte ni à une généralisation indépendante du contexte d'étude.

Annexes

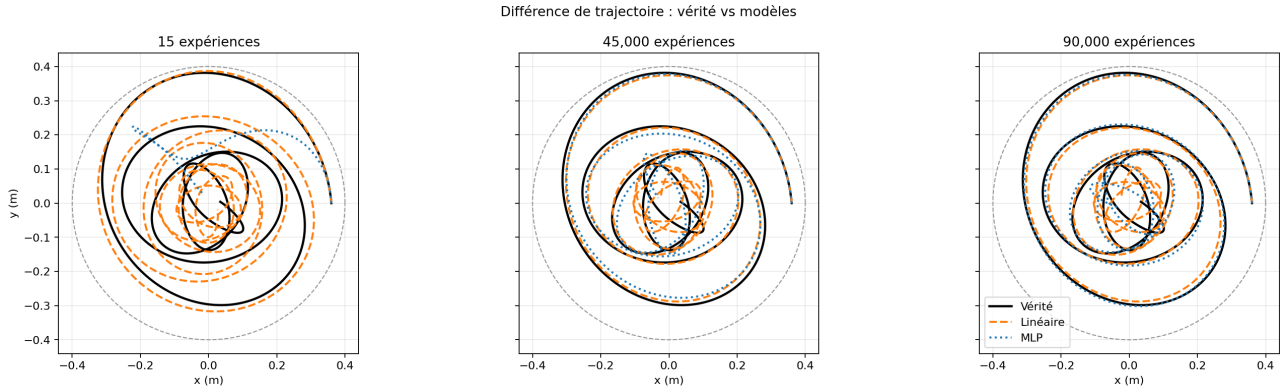

Figure 1: *Différence entre la vraie trajectoire et les modèless linéaire et MLP pour trois tailles de jeu de données (15, 45000, 90000).*

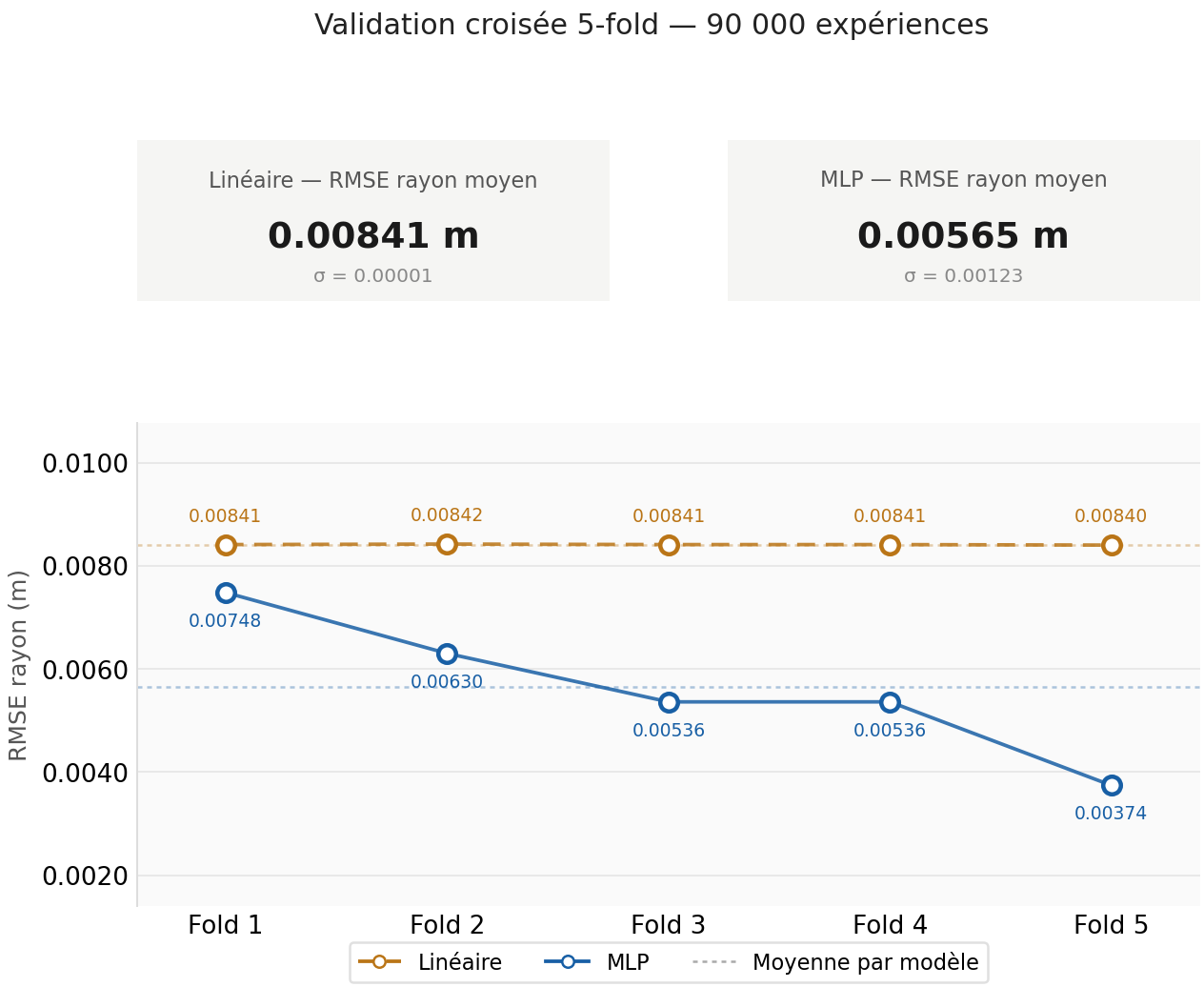

Figure 2: *Cross-validation avec K-fold (K=5) pour les modèles regression linéaire et MLP.*

References

[1] Maarten Jansen et al. *SYLLABUS Cours de Calcul Formel et Numérique*. INFO-F-205. Belgique, Feb. 2022.

[2] GeeksforGeeks. *Cross Validation in Machine Learning*. Sept. 2025. URL: https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/.

[3] Université Libre de Bruxelles. *Statistical Foundations of Machine Learning*. INFO-F422. 2025. URL: https://ulb-mlg.github.io/INFO-F422/practicals/_build/html/03.html.